Скаут

Платформа обратного найма ИТ-специалистов с верифицированным профилем ФСП

Спецтрек ФСП · ЛЦТ 2026 · Техническая документация

Репозиторий: github.com/Egor2809/fsp-scout

Техническая документация

Назначение и границы

Скаут связывает работодателей с анонимными профилями ИТ-специалистов. Доверенные сигналы отделены от заявленных: категория возникает из теста, достижения ФСП влияют на подбор при проверке и согласии кандидата, внешние резюме оцениваются без приписывания им этих сигналов. Документ описывает архитектуру, API, тест, подбор, развёртывание и валидацию.

Архитектура

React + TypeScript + Vite → `/api/v1` NestJS → PostgreSQL через Prisma. Keycloak выдаёт OIDC-токены по Authorization Code + PKCE; API проверяет JWT и JWKS. Redis/BullMQ обслуживает фоновую очередь. Отдельный worker проверяет код через Piston, назначает регулярные задания и обрабатывает удаление аккаунтов. PDF-резюме обрабатываются Python-инструментами в runtime-контейнере backend. Frontend раздаёт nginx; на публичном стенде Caddy завершает TLS.

API и данные

Источник контракта: `contracts/openapi.yaml`; матрица операций: `docs/api-matrix.md`. Входные тела проверяются Zod-схемами; ошибки возвращаются в формате RFC 9457. Снимки моделей frontend/backend проверяет `npm run check:contract` (232 файла моделей).

Основные области: профиль кандидата/компании, согласия, потребности, поиски, тесты и ответы, приглашения, вакансии/отклики, регулярные задания, достижения ФСП и внешняя оценка ранжирования. Схема хранения и SQL-миграции находятся в `backend/prisma`. Не заменяйте миграции командой `db push` при сдаче.

`POST /api/v1/eval/rank` принимает вакансии и резюме с идентификаторами, возвращает ранжирование и факторы. Нужна роль employer/admin. JSON Resume исключает поля имени, контактов и адреса из текстового подбора. CSV-выдача экранирует кавычки и нейтрализует формулы ячеек.

Тестирование кандидата

В исходниках реализованы выбор семейств CAT, 3PL/EAP с учётом угадывания, ограничения экспозиции, контроль компетенций, практика с кодом, серверная проверка ответов, последовательные состояния сессии и защита от повторных ответов. Ключи объективных заданий остаются на сервере. Необработанный код или свободный ответ не может автоматически стать подтверждённым верным ответом. Отсутствие или провал обязательной практики не заменяется высоким CAT-результатом.

Банк `expert-bank-20261010-v6`: 244 параметрических семейства — 160 с числовым ответом, 70 с выбором ответа (у каждого пул формулировок с разными правильными вариантами) и 14 канареек, по две на специализацию. Не меньше 17% заданий сессии — с выбором ответа (`choiceShare`). Параметры a и b — экспертные приоритеты; калибровка по реальным ответам предусмотрена после

пилота. Проверка песочницы требует реального Piston с установленными версиями рантаймов. Без Piston практика отключается, и категория подтверждается адаптивным блоком.

Подбор и объяснения

Публикация требует подтверждённую категорию и согласие. Учитываются скрытые компании, доступность, формат, город/релокация и обязательные навыки. Один грейд ниже запроса допускается со штрафом 15 баллов и пометкой в chips. Специализация остаётся строгой. Совпадение с выбранным грейдом имеет приоритет над соседними выбранными категориями. Итог ограничен диапазоном 0–100, отображаемые вклады суммируются в итоговую оценку.

Внутренний подбор использует покрытие навыков, Jaccard, отрасль, условия, перцентиль, подтверждённые навыки, регулярные задания и достижения ФСП. Внешний подбор использует покрытие словаря и BM25. Эти два режима различаются доступными доказательствами и не должны сравниваться как одинаковые модели.

Приватность и контроль доступа

Контакты раскрываются в контексте конкретной компании после согласия. Внешнее резюме не доказывает личность, тест или достижение. Подтверждение ФСП принимает подписанные webhook-данные с HMAC по исходным байтам. Удаление личности учитывает issuer и subject; worker обращается к Keycloak с отдельными служебными credentials. Шифрование контактов на уровне полей БД по пункту 4.5 плана ещё не внедрено.

Развёртывание

Локально: `docs/local-deployment.md`; публично: `docs/public-deployment.md`. Образы backend разделяют build/migrate/runtime. API и worker работают без root, с read-only файловой системой и временным `/tmp`. Compose ожидает PostgreSQL, Redis и завершение миграций. Готовность API проверяется `/api/v1/ready`; этот endpoint не заменяет отдельную проверку OIDC и Piston.

Валидация

Валидация воспроизводится командой `npm run eval` (backend), подробности в `docs/validation.md`. На 2000 синтетических кандидатах с независимым генератором ответов (3PL со сдвинутыми параметрами): точный грейд 57,0% (в продуктовом сценарии 62,7%), взвешенная к 0,81, ложные отказы 22,8%, r($\hat{\theta}$, θ) 0,87, test-retest к 0,71. При сливе ключей всех экземпляров $\hat{\theta}$ завышается на +0,62 против +2,68 у фиксированного теста. Подбор на отложенной выборке: nDCG@10 0,579 против 0,309 у BM25, P@10 0,669 против 0,361.

Какие модели, как и зачем

LLM не участвует в подтверждении навыков: грейд ставит только тест из экспертных семейств, ответы кандидатов модель не видит и не оценивает. Модель ускоряет одну задачу: написание черновиков новых заданий для банка.

Модель. GigaChat-2-Max через GigaChat API. Российская модель, как рекомендовал заказчик, данные не покидают российский контур. Модель задаётся флагом `--model`, запасная — GigaChat-2-Pro. YandexGPT подключается заменой HTTP-клиента в скрипте.

Как работает `npm run llm:drafts -- --count 14` (`backend/scripts/llm-draft-items.ts`):

1. Модель получает специализацию, грейд и список навыков из справочника банка. Персональных данных в запросе нет.
2. Числовое задание приходит как условие с параметрами, диапазоны и формула ответа. Эталон считает наш код по формуле, а не модель: формула разбирается по белому списку операций, проверяется на 200 случайных наборах параметров и сверяется с примерами модели.
3. Задание с выбором приходит как четыре формулировки одной ситуации с разными правильными ответами, как в боевом банке. Проверяется, что ответы различаются по сути, а `{n}` встроено в условие.
4. Слепая проверка: отдельным запросом модель решает экземпляр, не видя ключа. Расхождение с эталоном помечает черновик.
5. Рецензент: ещё один запрос ищет фактические ошибки ключа и неправдоподобные дистракторы.
6. Сходство с заданиями банка (Jaccard по словам) отсекает повторы.

Каждый черновик хранит происхождение: модель, версию и SHA-256 промпта, дату и `reviewed_by: null`. В боевой банк черновик попадает только после экспертной проверки, пилота и калибровки a и b. Автоматически туда не попадает ничего.

Первый прогон (`backend/llm-drafts/2026-10-11-GigaChat-2-Max.json`): 14 черновиков, 53 запроса, 15 485 токенов. Автоматические проверки отклонили 7: формулы с недопустимыми операциями, пропущенный `{n}`, замечания рецензента. 7 ждут эксперта, у 1 из них слепое решение разошлось с эталоном. Прошедшие проверки черновики тоже требуют вычитки: встречаются неточные ключи и слабые дистракторы. Поэтому модель здесь помощник автора, а не источник заданий. Банк `v6` из 244 семейств составлен экспертно, сгенерированных заданий в нём нет.

Сеть и ключ. API Сбера использует сертификат НУЦ Минцифры. Он лежит в `backend/certs/russian_trusted_ca_bundle.pem` и подключается только к запросам скрипта, в системное хранилище не устанавливается. Ключ `GIGACHAT_AUTH_KEY` хранится в `backend/.env`, этот файл не попадает в git.

Защита от фиктивных вакансий

- Вилка в приглашении и вакансии обязательна, верхняя граница не больше чем в 1,6 раза выше нижней. Вилка ниже ожиданий кандидата не отправляется, а порог кандидата компании не раскрывается.
- Квота приглашений на неделю: база плюс бонус до +10 за долю принятых приглашений за 90 дней и штраф –2 за каждое приглашение, отозванное после согласия кандидата. Квота не меньше 1.
- Кандидат видит надёжность компании в приглашении: число приглашений за 90 дней, долю принятых и число отзывов после согласия.
- Контакты кандидата раскрываются только компании, приглашение которой он принял.

Прокторинг

Во время теста браузер считает вставки из буфера и потери фокуса вкладки и отправляет их вместе с каждым ответом. Сервер сам измеряет время от выдачи задания до ответа. При завершении теста по этим данным строятся флаги `paste`, `focus_loss` и `fast_correct` (правильный ответ заметно

быстрее типичного для этого семейства). Флаги информационные: они не меняют грейд, потому что события браузера не доказывают нарушение. Ключи ответов хранятся только на сервере, у каждого кандидата свои параметры заданий.

Локальный запуск всего стека

Быстрый путь для проверки: `cd infra && cp .env.example .env && docker compose up --build`, затем http://localhost:8081. Сервис `seed` наполняет базу демо-данными, сервис `web` собирает фронтенд в режиме реального API и проксирует /api на api:8000. Демо-учётки (пароль `demo12345`) импортируются вместе с realm, их список в корневом README.md. Клиент `scout-e2e` с password grant нужен только для локальных сквозных проверок API.

В infra/compose.yaml добавлены migrate, api и worker. Dockerfile backend содержит отдельные стадии сборки, миграций и запуска. В runtime входят Python и pypdf для резюме; приложение работает пользователем node. API/worker имеют read-only файловую систему, временный /tmp, лимиты памяти/PID и сброшенные capabilities. Это локальная конфигурация: Keycloak работает в start-dev, TLS пока не настроен.

Из корня проекта:

```
if (-not (Test-Path infra/.env)) { Copy-Item infra/.env.example infra/.env }
# Заполните пароли и случайный FSP_WEBHOOK_SECRET длиной минимум 32 символа.
docker compose --env-file infra/.env -f infra/compose.yaml config --quiet
docker compose --env-file infra/.env -f infra/compose.yaml up --build -d
docker compose --env-file infra/.env -f infra/compose.yaml ps
docker compose --env-file infra/.env -f infra/compose.yaml logs --tail 100 migrate api
worker
```

В текущем шаблоне пароль PostgreSQL подставляется в URL подключения. Для этого локального шаблона используйте длинный случайный буквенно-цифровой пароль; пароли с URL-разделителями требуют отдельного URL-кодирования конфигурации. Не публикуйте .env или вывод конфигурации, содержащий секреты.

API: http://localhost:8000/api/v1, Keycloak: http://localhost:8080, Mailpit: http://localhost:8025. Frontend включён в Compose на http://localhost:8081, с VITE_API_MOCK=false и VITE_AUTH_MODE=oidc. Issuer токенов остаётся публичным localhost:8080, а серверный JWKS запрашивается через внутренний хост keycloak. Запуск контейнера Keycloak ещё не означает готовность его HTTP API: в первые секунды проверка JWT может не пройти, пока Keycloak не завершит запуск.

Миграции запускаются отдельным одноразовым сервисом; API/worker ждут его успешного завершения. Prisma db push не используется. /ready проверяет подключение к PostgreSQL и Redis, но не удостоверяет готовность Keycloak. Worker выполняет обслуживание, назначение регулярных заданий, скрытую оценку кода через настроенный Piston и удаление аккаунтов. LLM-проверка ещё не реализована.

Стек собирается и запускается по этой инструкции; та же конфигурация развёрнута на публичном стенде (public-deployment.md). Dockerfile и CI ставят зависимости через npm ci по lock-файлам.

После сборки запустите `powershell -ExecutionPolicy Bypass -File ./scripts/check-runtime.ps1`. Скрипт проверяет работающие контейнеры, успешное завершение migrate, /ready и discovery realm scout.

Фронтенд и аккаунты

Для разработки frontend можно запускать отдельно вместо контейнера. Сначала остановите сервис `frontend`, чтобы освободить порт 5173:

```
docker compose --env-file infra/.env -f infra/compose.yaml stop frontend
cd frontend
npm ci
npm run dev:real
```

Откройте http://localhost:5173. Команда dev:real работает на Windows и включает реальный API и OIDC. Регистрация создаёт кандидата; письмо подтверждения доступно в Mailpit на http://localhost:8025. Учётки из моков автоматически в Keycloak не создаются. Администратор Keycloak — admin, пароль находится в локальном infra/.env (KEYCLOAK_ADMIN_PASSWORD).

Для работодателя создайте пользователя в realm scout, подтвердите его email и назначьте realm role employer через административную консоль Keycloak. Свяжите его UUID с компанией операторской командой `backend/scripts/provision-employer.ts` (параметры --subject, --email, --company; DATABASE_URL и OIDC_ISSUER нужны в окружении). Скрипт запускается через tsx после установки backend-зависимостей и prisma:generate. Роль employer сама по себе компанию не создаёт; её отсутствие даёт 403 на корпоративных методах.

Запуск интерфейса пока не доказывает прохождение тестирования: требуется расширение банка, настройка ASSESSMENT_GRADE_POLICY и Piston; первые нормативные профили требуют отдельной проверки.

Публичный стенд с HTTPS

Конфигурация подготовлена для Linux VPS с Docker Compose. Нужны два DNS-имени, которые указывают на VPS: приложение и авторизация, например `scout.example.org` и `auth.example.org`. Для временного стенда можно использовать два имени вида `scout.<IP>.sslip.io` и `auth.<IP>.sslip.io`. Порты 80/TCP, 443/TCP должны быть доступны; сертификаты получает Caddy.

1. Создайте `infra/.env` из примера и замените все три секретных значения. Пароль PostgreSQL должен быть длинным случайным буквенно-цифровым значением: в этом шаблоне он используется внутри URL без дополнительного кодирования.
2. Добавьте в `infra/.env`:

```
dotenv APP_DOMAIN=scout.example.org AUTH_DOMAIN=auth.example.org
```

1. Подготовьте импорт realm для публичного адреса:

```
sh node scripts/prepare-public.mjs https://scout.example.org
```

1. Проверьте конфигурацию и запустите:

```
sh docker compose --env-file infra/.env -f infra/compose.yaml -f infra/compose.public.yaml config --quiet docker compose --env-file infra/.env -f infra/compose.yaml -f infra/compose.public.yaml up --build -d docker compose --env-file infra/.env -f infra/compose.yaml -f infra/compose.public.yaml ps
```

Миграции выполняются перед API. Frontend собирается с реальным API и публичным OIDC issuer. Keycloak работает в `start` с TLS на reverse proxy; административный путь закрыт на публичном Caddy. Для административного доступа используйте SSH туннель к loopback-порту 8080 на VPS. PostgreSQL, Redis, Mailpit и API также слушают только loopback; наружу открыт Caddy.

Realm импортируется при первом создании. Импорт не обновляет существующий realm: для прежней установки вручную измените redirect URI, web origins и logout URI клиента `scout-frontend`, прежде чем переходить на публичный адрес. Не удаляйте рабочие тома ради повторного импорта.

Приёмка

- Откройте `https://APP_DOMAIN/api/v1/ready`: ожидается успешный ответ.
- Откройте `https://AUTH_DOMAIN/realms/scout/.well-known/openid-configuration`: issuer должен точно совпадать с публичным URL realm.
- Пройдите вход кандидата и работодателя, проверьте callback и выход.
- Создайте работодателя и свяжите его с компанией по инструкции локального запуска.
- Настройте SMTP для реальной регистрации. Mailpit оставлен для демонстрационных учётных записей и не доставляет письма реальным адресатам.
- Настройте и проверьте банк, политику подтверждения и Piston перед демонстрацией полноценного теста; HTTPS сам по себе не устраняет ограничения банка.

Проверено на VPS 11.10.2026

Стенд `https://scout.141.133.201.48.sslip.io`: Ubuntu 24.04, 2 vCPU, 4 ГБ RAM, своп 4 ГБ. Для такой машины добавьте третий файл с ограничениями памяти Keycloak и PostgreSQL:

```
docker compose --env-file infra/.env -f infra/compose.yaml -f infra/compose.public.yaml -f infra/compose.vps.yaml up --build -d
```

Проверено: сертификат Let's Encrypt для обоих имён, `/api/v1/ready`, issuer Keycloak, закрытая снаружи административная консоль, вход кандидата и работодателя через ФСП ID (Keycloak, PKCE), согласия, опрос, экспресс-тест до результата, разбор потребности, подборка с объяснением, отправка инвайта, выполнение кода в Piston (Python, JavaScript). Память под нагрузкой одного проверяющего: около 1,5 ГБ из 3,8 ГБ, своп не используется.

Учётки жюри `jury1…5` создаются импортом realm. Чтобы вернуть им чистое состояние, остановите стек и удалите тома `scout_postgres` и `scout_redis`: при старте миграции, сид и импорт realm выполнятся заново, сертификаты Caddy сохранятся.

Подбор кандидатов

Категория и видимость

Поиск использует подтверждённую категорию теста, а не заявленную специализацию профиля. Нужны действующее согласие на публикацию, подтверждённый грейд и статус поиска работы. Скрытие от компании проверяется и в списках, и при прямом открытии карточки.

Начальная модель

Strength = 0.65 × P' + 0.15 × подтверждённые навыки + 0.10 × задания + 0.10 × полнота + бонус ФСП (0–15).

P' = 50 + (перцентиль – 50) × ρ, где ρ = τ² / (τ² + SE²), τ² = 0.8² / 12 — разброс θ внутри одной категории шириной 0.8. При SE ≈ 0.5 перцентиль внутри категории почти целиком шум, и его вес падает примерно до 0.2. На карточке показывается исходный перцентиль. Почему так, см. validation.md.

Fit = 100 × (0.25 × покрытие навыков + 0.45 × сходство словаря + 0.10 × отрасль + 0.20 × условия).

Score = clamp(0.6 × Fit + 0.4 × Strength – штраф точности, 0, 100).

Офлайн-сходство — Jaccard по идентификаторам навыков плюс смежность по таксономии: навык той же компетенции даёт 0,25 веса и объяснение «X вместо Y». Эмбеддинги не используются. Поле `semantic` сохранено для совместимости API, но evidence явно указывает выбранную метрику. Веса Fit и штраф за грейд ниже (15) подобраны покоординатным поиском на dev-выборке синтетики и проверены на holdout: nDCG@10 0,555 → 0,579. Отрасль и условия оставлены в формуле, хотя на синтетике не влияют на разметку: работодателю они важны. Сходство словаря (Jaccard) штрафует кандидатов, которые заявляют всё подряд. Веса собраны в `MATCH_WEIGHTS` (`backend/src/matching/matching.ts`); на экспертной разметке их нужно переобучить.

Подтверждённый навык весит в 1.5 раза больше заявленного. Желательные навыки имеют половинный вес. Пропущенные навыки уже уменьшают покрытие; их пояснение не добавляет второй штраф. Публикуемые вклады факторов суммируются до итогового скора с учётом округления и ограничения сверху.

ФСП учитывает только проверенные достижения с корректной прошедшей датой. При отсутствии истории отдельного отрицательного фактора нет.

Версии и shortlist

Каждое уточнение создаёт новую неизменяемую запись SearchVersion. Номер версии назначается под блокировкой строки Search. Shortlist связан с Search, а не SearchVersion. Изменение фильтров не удаляет shortlist. Если кандидат отозвал публикацию, запись сохраняется внутри БД, но перестаёт показываться работодателю.

Выдача для сохранённой версии пересчитывается по актуальным профилям: сохраняются условия поиска, но не замороженный снимок персональных данных.

Ограничения текущей реализации

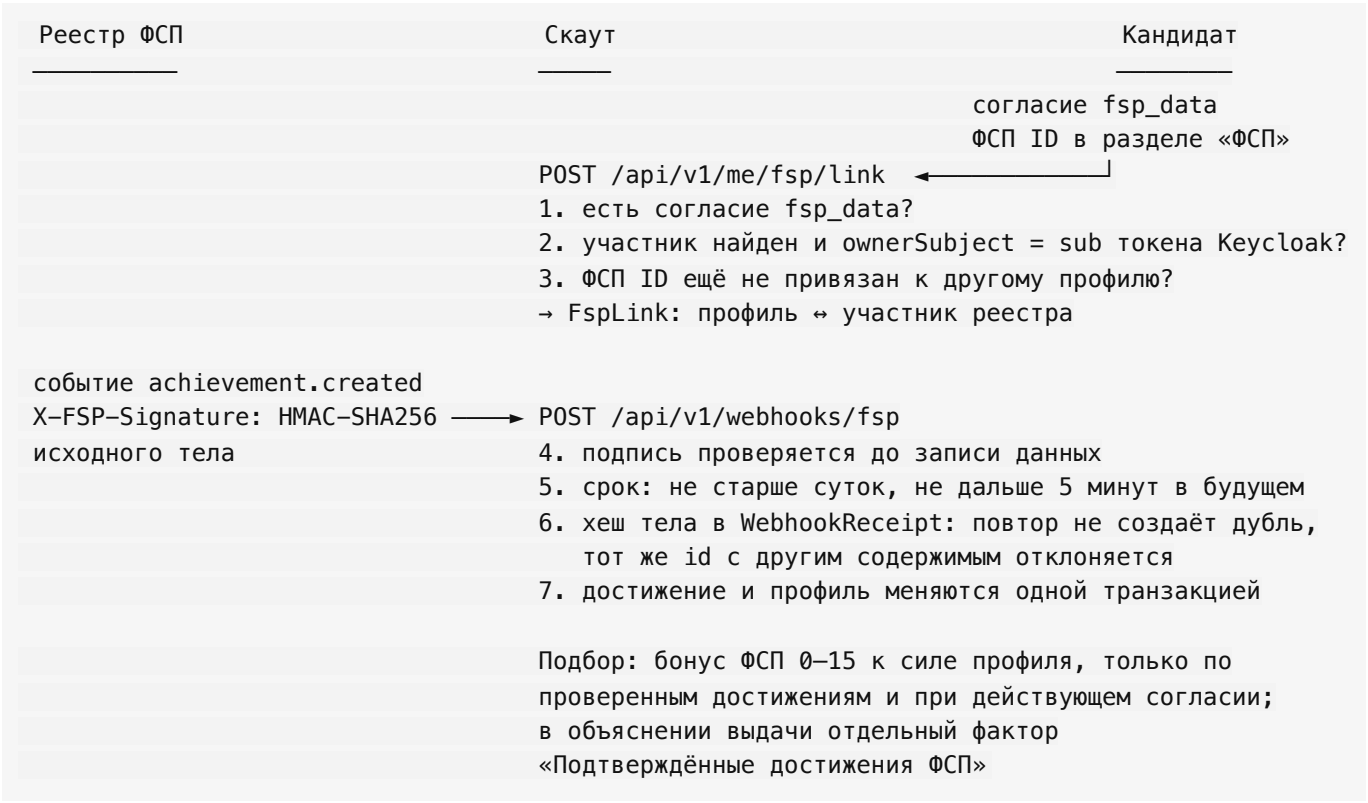
Ранжирование обрабатывает подходящих по специализации/грейду кандидатов в памяти после предварительной фильтрации PostgreSQL. Сложность линейна по размеру подходящей категории. Нагрузка и реальные пределы пока не измерены. Для больших категорий необходимы SQL-предфильтрация по навыкам/условиям и последующее ранжирование ограниченного множества.

Контакты раскрываются по отдельной записи Disclosure конкретной компании. Свободные поля опыта очищаются от известных контактов и типовых форматов. Это уменьшает риск утечки, но требует дополнительных интеграционных проверок, включая косвенные признаки личности в свободном тексте.

Интеграция с реестром ФСП

Реальный API ФСП организаторы не предоставили. В прототипе работает локальный реестр участников с тем же контрактом, что предлагается для боевой интеграции: привязка ФСП ID к профилю, подписанные события о достижениях и их учёт в подборе. Демонстрационные достижения синтетические и так помечены.

Схема



Что учитывается

- **Без истории ФСП нет штрафа.** Индекс и бонус равны 0, кандидат ранжируется по тесту и навыкам, в карточке пометка «Нет истории ФСП».
- **Грейд ФСП не меняет.** Категорию подтверждает только тест; достижения поднимают кандидата внутри категории.
- **Согласие отзывается.** Отзыв `fsp_data` удаляет связь и копию достижений из профиля.
- **Владение доказывается, а не заявляется.** Ввести чужой ФСП ID нельзя: привязка проходит только при совпадении владельца участника с проверенным `sub` OIDC-токена.

Сейчас в прототипе

Владельца участника и его достижения задаёт оператор через CLI с доступом к БД (`achievements.json` — массив достижений по схеме API; без файла создаётся участник без истории):

```
node --env-file=.env --import tsx scripts/provision-fsp.ts --subject KEYCLOAK_USER_UUID --fsp-id FSP-77-000001 --achievements achievements.json
```

Вебхук `POST /api/v1/webhooks/fsp` принимает `achievement.created`. Подпись — hex HMAC-SHA256 исходных байтов тела с секретом `FSP_WEBHOOK_SECRET` (не короче 32 символов), допускается префикс `sha256=`. Секрет подписи не даёт права создавать владельцев участников.

Что нужно от ФСП для боевой интеграции

1. **ФСП ID как провайдер входа в Keycloak.** Привязка по клиентскому account linking (`kc_action=idp_link`) уже заложена во фронтенде; операторский шаг CLI заменяется доказательством владения от доверенного IdP ФСП.
2. **Поток событий реестра.** Отправка `achievement.created` на вебхук с общим секретом; при необходимости — одноразовая серверная сверка участника, привязанная к сессии пользователя.
3. **Согласованный формат.** Транспорт, ключи, схема достижения и правила исправления протоколов задним числом согласуются с Федерацией после хакатона.

Валидация теста и подбора

Отчёт собирается одной командой из `backend`:

```
npm run eval   # ~40 с, seed 20261010, результат в backend/eval/validation-report.json
```

Скрипт `backend/eval/validate.ts` вызывает боевой код и ничего из него не копирует: выбор задания `selectFamily`, оценку `scoreCompletion` и `scoreOutcome`, контроль экспозиции `exposure`, банк `LIVE_BANK` с генерацией экземпляров `instantiate`, проверку ответа `gradeAnswer`, подтверждение навыков `confirmedSkills`, перцентиль `empiricalPercentile`, фильтры и скоринг подбора `hardFilters` и `explain`, текстовый ранжировщик `/eval/rank` (`rankExternal`), метрики `rankingMetrics`. Цифры в этом документе взяты из отчёта и вручную не правились. Старая симуляция `eval/simulate.ts` удалена: в ней ответы генерировала та же модель, которой их потом оценивали, а банк был выдуманным.

Методика

Всё синтетическое. Реальных кандидатов с известным «настоящим» уровнем у нас нет, как и у конкурентов. Поэтому отчёт проверяет, как алгоритм ведёт себя на популяции со скрытой истиной. Это не доказательство того, что тест работает на живых людях.

Генератор ответов не совпадает с оценщиком. Оценщик знает только экспертные `a = 1,2` и `b` из банка и считает EAP по 3PL: угадывание c = 0 у числовых, 1/число вариантов у single_choice, 0,05 у multiple_choice. Ответы генерирует 3PL с другими параметрами семейства: `a_true = a × U(0,7; 1,3)`, `b_true = b + U(−0,4; 0,4)`, угадывание `c = 0,02` у числовых (в банке `v6` 160 числовых семейств, 70 с выбором ответа и 14 канареек; для выбора `c` как у оценщика). Реальный сдвиг: |Δa|/a в среднем 0,16, |Δb| 0,20. Оценщик об этих сдвигах не знает.

Популяция теста. 2000 кандидатов, θ ~ N(0,1), поровну по 7 специализациям. Истинный грейд определяется по боевым порогам: ниже −1,2 — «ниже junior» (240 человек), junior 453, middle 642, senior 452, lead 213. Заявленный грейд: 60% указывают истинный, 20% на ступень выше, 20% на ступень ниже. Сессии идут подряд с общим счётчиком экспозиции, как в продакшене: потолок 0,4, окно считается по специализации. Политика теста боевая: 8–14 заданий, SE ≤ 0,4, top-3, обязательное покрытие компетенций, в которых у специализации не меньше трёх семейств. Практический блок с кодом не моделируется, `practiceTotal = 0`, как при выключенной песочнице.

Грейд по $\hat{\theta}$ — это полоса, в которую попала оценка. Пять классов: «ниже junior» и четыре грейда. **Решение теста** — подтверждён ли заявленный грейд (`outcome ≠ failed`, то есть P(θ ≥ порог) ≥ 0,6). **Продуктовый сценарий**: кандидат идёт по подсказкам системы. Провалил — пересдаёт ступенью ниже, получил `passed_confident` — пробует ступенью выше. Всего не больше четырёх попыток. Итоговый грейд — последний подтверждённый.

ДИ везде — перцентильный бутстрэп, 1000 повторов, 95%. Для теста ресэмплируются кандидаты, для подбора — потребности.

Тест

Метрика (N = 2000)	Значение	ДИ 95%
Точный грейд по $\hat{\theta}$ (5 классов)	**57,0%**	54,8–59,1
Грейд ±1	**98,2%**	97,7–98,8
к Коэна, квадратичная	**0,811**	0,796–0,825
Корреляция $\hat{\theta}$ и θ (Пирсон)	**0,871**	0,861–0,881
RMSE $\hat{\theta}$	0,488	0,472–0,505
Средняя длина / средний SE	14 заданий / 0,45	
Ложный отказ: грейд ≥ заявленного, но не подтверждён	**22,8%**	20,6–24,9
Ложное подтверждение: грейд ниже заявленного, но подтверждён	23,8%	20,5–27,6
Точность решения «подтвердить / нет»	76,9%	74,9–78,8
Итоговый грейд в продуктовом сценарии, точно	62,7%	60,6–64,8
Итоговый грейд в продуктовом сценарии, ±1 / к	99,6% / 0,845	99,2–99,8 / 0,833–0,856
Фиксированная форма из 12 заданий того же банка, точно / к / r	52,4% / 0,765 / 0,833	

Порог уверенности `passProbability` = 0,6. Его подобрали на отдельном seed 777 по сетке 0,5–0,7: при 0,7 ложные отказы 32,9% против 16,8% ложных подтверждений, при 0,6 ошибки уравниваются (24,6% и 22,9%), а точность решения растёт с 71,5% до 75,9%. На основном seed результат на банке `v3` подтвердился: 22,9% и 22,2%, точность 77,3%. На банке `v4` с заданиями выбора и честным угадыванием 25% при оценке по 2PL: 20,8% и 29,5%, точность 76,9%. После перехода оценщика на 3PL (банк `v6`) и квоты заданий с выбором: 22,8% и 23,8%, точность 76,9%. Сами пороги грейдов не настраивались: они задают истинный грейд в симуляции, и их подбор был бы порочным кругом. Таблицы ниже посчитаны на банке `v6` при пороге 0,6.

Квота заданий с выбором. С учётом угадывания задания с выбором менее информативны, и CAT выдавал их в 1,8% случаев при 29% банка. Теперь не меньше 17% заданий сессии — с выбором ответа: 2 из 14 в полном тесте, 1 из 6 в экспрессе. Цена: точный грейд 58,5% → 57,0%, ложные подтверждения 23,3% → 23,8%. Без квоты кандидат почти не видит заданий с выбором, и их устойчивость к сливу ничего не даёт.

Что исправлено по итогам валидации

Сессия больше не обрывается на обязательном покрытии. Раньше она требовала все компетенции банка, в том числе те, где одно-два семейства, а потолок экспозиции 0,4 их быстро исчерпывал: 48% сессий заканчивались `coverage_unavailable` (409 на `/next`), точный грейд был 51,6%, ложных отказов 64%. Теперь обязательны только компетенции, где семейств не меньше `ceil(1 / maxExposure) = 3` (`requiredCompetencies` в `live-bank.ts`). Если покрытие всё равно упирается в экспозицию, компетенция снимается с обязательных для этой сессии (`selectWithWaiver`, поле `waivedCompetencies` в политике сессии). Так случилось в 51 сессии из 2000, и все они дошли до 14 заданий.

Test-retest

Тот же кандидат, тот же заявленный грейд, другой seed сессии, отдельная волна из 2000 сессий.

Метрика	Значение	ДИ 95%
Совпадение грейда по $\hat{\theta}$	48,3%	46,2–50,5
к квадратичная	**0,712**	0,689–0,733
Совпадение решения (подтверждён / нет)	73,2%	71,2–75,1

Метрика	Значение	ДИ 95%
$r(\hat{\theta}_1, \hat{\theta}_2)$	0,762	0,743–0,779
Общих семейств в двух сессиях, в среднем	5,3 из 12	

Слив банка

600 «слабых» кандидатов (θ ниже порога middle, в среднем –1,10) заявляют middle. Любое подтверждение у них ложное. Утекает k% семейств банка специализации. Сценариев два.

- **Утечка ключей экземпляров.** Злоумышленник собрал по 300 реальных экземпляров каждого утёкшего семейства с ответами (генерируются настоящим `instantiate`). Если формулировка совпала дословно, он берёт ответ из дампа, иначе вписывает самый частый ответ дампа. Ответ проверяет боевой `gradeAnswer`.
- **Утечка метода решения.** Кандидат знает, как решать утёкшие семейства, и решает их верно.

Фиксированный тест — те же 12 заданий с постоянными числами для всех, поэтому для него утечка ключа и утечка метода совпадают. Завышение $\hat{\theta}$ считается относительно того же кандидата без утечки.

Утечка	CAT: ключи экземпляров	CAT: метод решения	Фиксированные 12
нет — ложные подтверждения	15,0%	15,0%	23,3%
25% — завышение $\hat{\theta}$ / ложные подтв.	+0,13 / 23,3%	+0,66 / 53,8%	+0,64 / 71,8%
50%	+0,27 / 31,2%	+1,45 / 86,5%	+1,24 / 100,0%
100%	**+0,62** / 53,3%	+3,14 / 100,0%	**+2,68** / 100,0%

ДИ для каждой ячейки есть в отчёте. Выводы:

- Параметризация режет вред от слитых ключей вчетверо: +0,62 против +2,68 у фиксированной формы при полном сливе, +0,27 против +1,24 при сливе половины. На банке `v3` без заданий с выбором было +0,64, на `v4` +0,97: ключи заданий с выбором переносились между экземплярами. В `v6` у них пулы формулировок с разными правильными вариантами.
- От утечки метода параметризация не защищает, и не должна: знать метод и значит уметь решать.
- **Дамп банка.** В банке `v4` правильный вариант в заданиях с выбором был одинаков во всех экземплярах, и ответ по дампу из 300 экземпляров угадывался в среднем в 52% случаев. В `v6` у каждого семейства с выбором пул формулировок с разными правильными вариантами: средняя угадываемость 26%, выше 50% — у 48 из 244 семейств (было 115). Остались семейства со 100% угадыванием, например `live-srcset-choice`, `live-closure-var-let`, `live-cidr-hosts`, `live-touch-target`, `live-decision-table`.

Канарейки

14 канареечных семейств (по 2 на специализацию) ставят флаг `leak_suspected`, если канарейка решена верно и аномально быстро ($z < -2{,}5$ по логнормальной модели времени), а оба соседних обычных задания решены неверно. Флаг не меняет грейд. Модель времени в симуляции: медиана семейства 45 с, разброс $\sigma = 0{,}5$ в логарифме, ответ по готовому решению в 8 раз быстрее.

Сценарий	Сессий с канарейкой	Флаг
Честные кандидаты	43,5%	0,0%

Сценарий	Сессий с канарейкой	Флаг
Слив ключей экземпляров, 100%	50,3%	0,0%
Готовый решатель для 25% семейств	55,8%	4,7%
Готовый решатель для 50% семейств	53,2%	2,5%
Готовый решатель для всех семейств	44,3%	0,0%

Ложных срабатываний нет, но ловит правило мало. Слитые ключи экземпляров к новым экземплярам канарейки не подходят, поэтому дамп канарейку не решает вовсе. Готовый решатель для всех семейств не оставляет неверных соседей. Правило консервативное: оно отмечает только явные случаи для проверки человеком.

Экспозиция

2000 сессий первой волны, 14 заданий в каждой:

Специализация	Семейств	Макс. доля сессий на семейство	Средняя доля
backend	66	0,399	0,212
frontend	54	0,399	0,259
mobile	50	0,399	0,280
qa	52	0,399	0,269
devops	59	0,399	0,237
data_ml	57	0,400	0,246
analyst	56	0,400	0,250

Потолок 0,4 соблюдается, неиспользованных семейств нет.

Подбор

Данные. Два независимых мира: dev для подбора формулы и holdout для отчёта. В каждом 1750 кандидатов (по 250 на специализацию) и 210 потребностей работодателей. У кандидата скрыты θ, набор навыков, которыми он реально владеет (4–7 из 14 самых частых навыков специализации), и уровень владения каждым (θ + N(0; 0,6)). Наблюдаемое резюме шумное: 10% реальных навыков не указаны, четверть кандидатов «накручивает» резюме (3–5 лишних навыков, грейд на ступень выше, повторы ключевых слов). Каждый кандидат проходит **боевой тест** по продуктовому сценарию. Ответы зависят от θ и от владения навыками, привязанными к семейству, поэтому тест в этой части многомерный, а оценщик нет. Из теста берутся подтверждённая категория, перцентиль внутри категории (`empiricalPercentile`), SE и подтверждённые навыки. ФСП-достижения и регулярные задания тоже генерируются из θ с шумом.

Золотая релевантность считается только по скрытой истине и не использует наблюдаемые поля формулы: ни зарплату, ни отрасль, ни формат, ни заявленные навыки. Метка 0–3 равна `соответствие грейда × (0,7 × доля must-навыков + 0,3 × доля nice-навыков)` . Навык засчитывается, если им реально владеют на уровне не ниже порога грейда – 0,4. Соответствие грейда в основной разметке: совпал — 1, на ступень выше — 0,85, на две выше — 0,6, на ступень ниже — 0,3. Релевантен тот, у кого метка ≥ 2. Для P@10 и MRR@10 берётся именно этот порог, nDCG@10 считается по градуированной метке. Порог переквалификации — произвольное решение авторов, поэтому приведены ещё две разметки: строгая (выше на ступень 0,7, на две 0) и мягкая (любой грейд не ниже нужного — 1). Честно о порядке работы: первой была строгая разметка. После первого прогона стало видно, что вывод зависит от штрафа за переквалификацию. Тогда основной

сделали промежуточную, строгую оставили и добавили мягкую. Формулу подбора меняли уже после этого, глядя только на dev.

Системы: * **наш подбор** — боевые `hardFilters` + `explain`; * **BM25** (k1 = 1,2, b = 0,75, реализован в скрипте) по тексту резюме, в том числе заявленный грейд, на всём пуле специализации; * **BM25 + наши фильтры** — тот же пул, что видит наш подбор, но в порядке BM25. Так видно, что даёт именно формула скоринга; * **/eval/rank** — наш текстовый ранжировщик для внешних резюме; * **случайный порядок**.

Holdout, основная разметка, 210 потребностей:

Система	P@10	nDCG@10	MRR@10
Наш подбор	**0,669** (0,641–0,697)	**0,579** (0,559–0,597)	**0,896** (0,864–0,927)
BM25 + наши фильтры	0,482 (0,454–0,511)	0,417 (0,393–0,442)	0,634 (0,588–0,679)
BM25	0,361 (0,333–0,391)	0,309 (0,284–0,334)	0,558 (0,509–0,606)
/eval/rank (только текст)	0,391 (0,365–0,417)	0,331 (0,308–0,356)	0,553 (0,503–0,606)
Случайный	0,216	0,157	0,390

Парные разности на holdout: * наш подбор – BM25: nDCG +0,270 (0,246…0,293), P@10 +0,308 (0,277…0,339); * наш подбор – BM25 на том же пуле: nDCG +0,162 (0,140…0,183), P@10 +0,187 (0,158…0,214);

Выигрыш у BM25 складывается из двух частей: пул (фильтры по категории с допуском на грейд ниже и must-навыкам) и сама формула скоринга. На одном и том же пуле формула обходит BM25 на +0,128 nDCG: она учитывает подтверждённую категорию, перцентиль, ФСП и смежные навыки.

`/eval/rank` для внешних резюме обходит простой BM25: nDCG@10 0,331 против 0,309, P@10 0,391 против 0,361. Раньше он проигрывал (0,292): навыкам из словаря давалось 60% веса, а заявленный грейд почти не учитывался. Теперь веса такие: навыки 37,5, BM25 25, грейд из резюме 25 (на грейд ниже — сильный штраф, переквалификация — слабый), условия 10, отрасль 2,5. Веса выбраны на dev по сетке, цифры выше — holdout.

Абляции (holdout)

Без сигнала	P@10	nDCG@10	Разность с полной моделью: nDCG / P@10
— (полная)	0,669	0,579	
без теста	0,450	0,438	+0,141 (0,120…0,164) / +0,220 (0,195…0,243)
без ФСП	0,665	0,580	–0,001 (–0,006…0,004) / +0,004 (–0,003…0,011)
без регулярных задач	0,662	0,579	–0,000 (–0,006…0,005) / +0,007 (–0,002…0,016)

«Без теста» значит, что категория берётся из заявленного в резюме грейда, перцентиль равен 50 для всех, подтверждённых тестом навыков нет, видны все кандидаты. Такого режима в продукте нет: без теста кандидат в поиск не попадает.

- **Тест** — главный сигнал: +0,14 nDCG и +0,22 P@10 к ранжированию по заявленному грейду. По строгой и мягкой разметкам вывод тот же (nDCG 0,536 против 0,437 и 0,618 против 0,432). Подтверждённый грейд всё ещё чаще занижен, чем завышен (holdout: верный у 785, на ступень ниже у 563, на две ниже у 36, выше у 98). Поэтому жёсткие фильтры прячут 65% релевантных кандидатов, и это главный резерв.
- **ФСП** — вклад в пределах шума. На банке v3 множитель 0,3 давал +0,011 P@10 (значимо), на банках v4 и v6 — около нуля (на v6 nDCG –0,001, интервал –0,006…0,004; P@10 +0,004,

интервал –0,003…0,011). Достижения ФСП в генераторе коррелируют с θ, и их информация почти полностью дублирует тест. Пробовали усиливать ФСП при неточном тесте (бонус × (1 – 0,6 × надёжность перцентиля)): +0,003 nDCG на dev и +0,001 на holdout, незначимо, поэтому не взяли. Бонус ФСП оставлен как объяснимый сигнал внутри категории: он нужен там, где теста мало, а в симуляции тест есть у всех.
- **Регулярные задачи** — по nDCG вклад нулевой (+0,002, незначимо), по P@10 +0,013 (значимо). В симуляции задачи зависят только от θ и активности, а не от навыков потребности.

Подбор весов формулы

Веса `MATCH_WEIGHTS` (`backend/src/matching/matching.ts`) подобраны покоординатным поиском по сетке на dev, holdout в подборе не участвовал. Отрасль и условия не опускали ниже 0,1 и 0,2: синтетическая разметка их не учитывает, а работодателю они нужны.

Вес	Было	Стало
Покрытие навыков в Fit	0,40	0,25
Сходство словаря навыков (Jaccard) в Fit	0,25	0,45
Отрасль в Fit	0,15	0,10
Штраф за грейд ниже запроса	8	15
nDCG@10 dev / holdout	0,593 / 0,555	0,593 / 0,579

Jaccard штрафует тех, кто заявляет всё подряд: лишние навыки увеличивают объединение множеств. Пробовали показывать кандидатов на грейд выше со штрафом: +0,007 nDCG на holdout, но это меняет смысл поиска для работодателя, поэтому не взяли.

Смежные навыки по таксономии

Если у кандидата нет нужного навыка, но есть навык той же компетенции из таксономии (например, другой фреймворк того же слоя), он получает 0,25 от веса навыка (0,375 для подтверждённого) и объяснение «X вместо Y». Вес выбран на dev по сетке 0 / 0,15 / 0,25 / 0,4. Эффект на синтетике нейтральный (±0,002 nDCG): генератор выбирает навыки кандидата независимо от компетенций, поэтому смежность в нём не несёт сигнала. На реальных стеках навыки одной компетенции связаны, и эффект нужно проверить на пилоте.

Изменение формулы (одно)

Цифры этого раздела получены до исправления покрытия: изменение формулы делалось раньше.

Первый прогон показал, что внутри пула, прошедшего фильтры, наша формула упорядочивает кандидатов **хуже** BM25: на holdout nDCG 0,372 против 0,397. Больше всего мешал перцентиль. На dev его корреляция с золотой меткой внутри пула отрицательная (–0,10), а весит он 26 баллов из 100.

Объяснение простое. Внутри одной категории шириной 0,8 истинный разброс θ равен $\tau^2 = 0{,}8^2/12 \approx 0{,}053$, а ошибка теста $SE^2 \approx 0{,}25$. Надёжность внутрикатегорийного перцентиля $\rho = \tau^2/(\tau^2 + SE^2) \approx 0{,}2$, то есть примерно на 80% он шум. Изменение в `strength()` (`src/matching/matching.ts`): перцентиль сжимается к 50 с коэффициентом ρ, который считается по SE самого кандидата. Свободных параметров нет, τ берётся из ширины грейда. На карточке и в объяснении по-прежнему показан исходный перцентиль, в evidence добавлен множитель.

	dev, до → после	holdout, до → после
nDCG@10	0,370 → 0,403	**0,372 → 0,409**
P@10	0,488 → 0,496	0,503 → 0,516
MRR@10	0,742 → 0,801	0,750 → 0,795
nDCG@10, строгая разметка	0,252 → 0,311	0,247 → 0,305
nDCG@10, мягкая разметка	0,502 → 0,498	0,515 → 0,518

Решение принималось по dev, holdout подтвердил результат. По мягкой разметке, где переквалификация не штрафуется, изменение нейтрально. Цифры «до» получены на коде коммита 24c1668. Чтобы их воспроизвести, верните прежний `strength()` и запустите `npm run eval`. Тесты обновлены: добавлена проверка, что при меньшем SE перцентиль весит больше.

Сравнение с конкурентами

У всех ниже синтетика с собственными генераторами, порогами и разметкой. Цифры показывают порядок величин, а не результат на общем наборе данных.

Что	Конкурент	Мы
Точный грейд	Shaurma 76% (2000 канд.)	57,0% (5 классов, полоса 0,8)
κ	Shaurma 0,87	0,811
Test-retest κ	Shaurma 0,864	0,712
$r(\hat{\theta}, \theta)$	andercat2 0,921	0,871
Ложные отказы	andercat2 22,5%	22,8%
Слив банка: завышение	Shaurma –0,10 (CAT) против +1,32 (фикс.)	+0,27 при 50% ключей, +0,62 при 100%; фикс. +1,24 / +2,68
P@10 против BM25	Shaurma 0,855 против 0,35	0,669 против 0,361
nDCG@10 против BM25	diktator06 0,591 против 0,282	0,579 против 0,309
Вклад ФСП (абляция)	ctrl-alt-win измеряет	P@10 +0,004, незначимо

Где мы честно отстаём: точность грейда и повторяемость. Главная причина — 14 заданий и SE ≈ 0,45. Где мы сопоставимы или сильнее: независимость генератора (истинные параметры отличаются от банковских, утечки моделируются через настоящий генератор экземпляров) и честная абляция ФСП. Отрыв от BM25 по подбору у нас меньше, чем заявляют Shaurma и diktator06. Наша разметка не использует поля, на которых работает формула, и BM25 в ней получает тот же заявленный грейд в тексте.

Ограничения и что не проверено

- В банке ещё 48 семейств, ответ которых по дампу из 300 экземпляров угадывается чаще чем в половине случаев (см. «Слив банка»). Канарейки ловят только явные случаи.
- Нет ни одного реального кандидата. Экспертные `b` банка не откалиброваны, а истинные параметры в симуляции — это экспертные плюс шум, не данные.
- В части «Тест» модель одномерная, как у оценщика. Тест получается проще реальности.
- Практическое задание с кодом и вопрос о подходе не моделируются, хотя в продакшене при включённой песочнице они влияют на итог.
- Нет временных эффектов: окна экспозиции в 90 дней, кулдаунов пересдачи, параллельных сессий.

- Разметку подбора придумали мы. Вывод о вкладе теста и ФСП зависит от того, как штрафовать переквалификацию, поэтому приведены три разметки.
- Поведение накрутчиков упрощено: копирование чужих резюме, подмена личности и списывание по ходу теста не моделируются.
- Зарплата, отрасль и формат в разметку не входят. Их вклад в формулу этот отчёт не оценивает и скорее штрафует как шум.

Библиотеки и компоненты

Версии ниже совпадают с `backend/package.json`, `frontend/package-lock.json`, `backend/requirements.txt`, Dockerfile и `infra/compose.yaml`. Зависимости ставятся через `npm ci` по lock-файлам, поэтому сборка воспроизводима.

Инфраструктура

Компонент	Версия	Роль	Лицензия
Node.js	24 (`node:24-bookworm-slim`)	среда выполнения API, worker и сборки фронтенда	MIT
PostgreSQL	17.6	основное хранилище	PostgreSQL License
Redis	7.4.5	очередь BullMQ для скрытой оценки кода и фоновых задач	RSALv2 / SSPLv1
Keycloak	26.3.4	вход OIDC (Authorization Code + PKCE), роли, подтверждение почты; совместим с будущим ФСП ID	Apache-2.0
Piston	`ghcr.io/engineer-man/piston`	изолированный запуск кода кандидата: Python 3.10.0, JavaScript (Node) 18.15.0	MIT
nginx	1.27.5	раздача фронтенда и прокси `/api`	BSD-2-Clause
Caddy	2	HTTPS и сертификаты Let's Encrypt на публичном стенде	Apache-2.0
Mailpit	1.27.8	почта подтверждения регистрации на стенде	MIT
Docker Compose	v2	запуск всего стека одной командой	Apache-2.0

Redis начиная с 7.4 распространяется под RSALv2/SSPLv1. Протокол и команды совместимы с Valkey 8 (BSD-3-Clause); замена образа не требует изменений кода.

Бэкенд

Библиотека	Версия	Назначение	Лицензия
NestJS (`@nestjs/common`, `core`, `platform-express`)	11.1.6	HTTP API, модули, guards	MIT
Prisma (`prisma`, `@prisma/client`)	6.16.2	схема БД, миграции, доступ к PostgreSQL	Apache-2.0
BullMQ	5.58.5	очередь оценки кода, повторы, аренды задач	MIT
jose	6.1.0	проверка JWT Keycloak по JWKS	MIT
Zod	4.1.11	проверка тел запросов и ответов по контракту OpenAPI	MIT
RxJS	7.8.2	зависимость NestJS	Apache-2.0
reflect-metadata	0.2.2	декораторы NestJS	Apache-2.0
TypeScript	5.9.2	язык бэкенда	Apache-2.0
tsx	4.20.5	запуск TypeScript-скриптов (сид, утилиты)	MIT
ReportLab (Python)	5.0.1	PDF-профиль кандидата с QR-ссылкой проверки	BSD
pypdf (Python)	6.19.0	извлечение текста из PDF-резюме	BSD-3-Clause

Фронтенд

Библиотека	Версия	Назначение	Лицензия
React, React DOM	19.3.0	интерфейс обоих кабинетов	MIT
React Router	8.4.0	маршрутизация	MIT
TanStack Query	5.104.1	запросы к API и кеш	MIT
oidc-client-ts	3.5.0	вход через Keycloak (PKCE)	Apache-2.0
Zod	4.6.5	проверка форм и ответов	MIT
Tailwind CSS	4.3.3	стили	MIT
Radix UI	1.6.7	доступные компоненты (диалоги, меню, списки)	MIT
cmdk	1.1.1	выбор навыков с поиском по синонимам	MIT
CodeMirror (`codemirror`, языки Python, JavaScript, Go)	6.0.2 / 6.2.1 / 6.2.5 / 6.0.1	редактор кода в практическом задании	MIT
Recharts	3.10.1	графики результата теста и механики	MIT
Motion	14.0.0	анимации	MIT
Sonner	2.0.8	уведомления	MIT
Tabler Icons	3.48.0	иконки	MIT
Fontsource (Wix Madefor, Source Serif 4, JetBrains Mono и др.)	5.3.0	локальные шрифты без внешних CDN	OFL-1.1
Vite	8.3.2	сборка	MIT
TypeScript	6.0.3	язык фронтенда	Apache-2.0

Разработка и проверки

Инструмент	Версия	Назначение	Лицензия
Orval	8.40.0	генерация клиента, типов и моков из OpenAPI	MIT
MSW	3.0.2	моки API для разработки и e2e	MIT
Vitest	4.1.11	модульные тесты фронтенда	MIT
Playwright	1.63.0	сквозные тесты в браузере (десктоп и 375 px)	Apache-2.0
ESLint	10.12.0	статический анализ фронтенда	MIT
Node test runner	встроен в Node.js 24	модульные тесты бэкенда, валидация `npm run eval`	MIT
GitHub Actions	—	CI: сборка, тесты, снимок контракта, валидация, e2e	—

Внешние сервисы

Сервис	Где используется	Обязателен
GigaChat-2-Max (Сбер)	только черновики новых заданий (`npm run llm:drafts`); ответы кандидатов модель не видит	нет, платформа работает без ключа